api-gateway — Teknik Genel Bakış

Proje	Servis	Dil / Runtime	Son Güncelleme
Tresaurio Kripto Portföy Yönetim Platformu	api-gateway	Go 1.26	2026-04-13

1. Servisin Amacı

api-gateway, Tresaurio platformunun tüm client isteklerini karşılayan tek giriş noktasıdır. Gelen HTTP ve WebSocket isteklerini downstream servislere yönlendirir; kimlik doğrulama, hız sınırlama ve hata yönetimi gibi cross-cutting concern'leri merkezi olarak uygular.

Downstream servisler bu gateway'i görmeden doğrudan dış dünyaya açılmaz; tüm trafik buradan geçer.

Sorumluluk	Açıklama
Reverse Proxy	İstekleri market-data ve exchange-service'e yönlendirir
JWT Doğrulama	Korumalı endpoint'lerde token doğrular, `X-User-ID` header'ını downstream'e iletir
Rate Limiting	IP bazlı istek sınırlaması — saniyede 10, burst 30
Circuit Breaker	Downstream servis art arda 5 hata verirse devreyi açar, 503 döner
Health Aggregation	Tüm downstream servislerin durumunu tek endpoint'ten sunar
WebSocket Proxy	WebSocket bağlantılarını upstream servise tüneller
Swagger UI	Tüm API endpoint'lerini tarayıcıdan interaktif olarak belgeler ve test imkânı sunar

2. Mimari

```
Client (HTTP / WebSocket)
        │
        ▼
┌──────────────────────────────────────────────────────┐
│              api-gateway (port 9000)                 │
│                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐ │
│  │ Rate Limiter │  │ JWT Auth     │  │ Logger /     │ │
│  │  (global)    │  │(protected    │  │ Recoverer    │ │
│  │              │  │  routes)     │  │              │ │
│  └──────────────┘  └──────────────┘  └──────────────┘ │
│         │                │                           │
│  ┌──────▼────────────────▼──────────────────────┐ │
│  │              chi Router                      │ │
│  │                                              │ │
│  │  /health           → Health Handler          │ │
│  │  /api/v1/quotes/*  → CircuitBreaker          │ │
│  │  /ws/quotes/*      → WebSocket Proxy         │ │
```

```
│   │  /positions/*      → CircuitBreaker + JWT    │ │
│   │  /api/v1/pnl/*     → CircuitBreaker + JWT    │ │
│   │  /api/v1/orders/*  → CircuitBreaker + JWT    │ │
│   │  /ws/positions/*   → WebSocket Proxy + JWT   │ │
│   └──────────────────────────────────────────────┘ │
└────────────────────────────────────────────────────┘
            │                        │
            ▼                        ▼
    ┌─────────────────┐      ┌──────────────────────┐
    │  market-data    │      │   exchange-service   │
    │  (ngrok URL)    │      │   (ngrok URL)        │
    └─────────────────┘      └──────────────────────┘
```

Middleware Zinciri

Her istek aşağıdaki sırayla middleware'den geçer:

```
Request
   │
   ├─► CORS            (Access-Control-* headers; OPTIONS preflight → 204)
   ├─► Logger          (chi — tüm istekleri loglar)
   ├─► Recoverer       (chi — panic'leri yakalar, 500 döner)
   ├─► RateLimiter     (IP bazlı token bucket)
   ├─► Metrics         (Prometheus sayaçları ve histogram)
   ├─► RequestLogger   (method, path, status, latency, IP, userID)
   ├─► [JWTAuth]       (sadece korumalı route group'larında)
   ├─► CircuitBreaker  (market-data veya exchange CB'si)
   └─► Proxy Handler   (reverse proxy / websocket proxy)
```

---

3. Bileşen Detayları

3.1 Reverse Proxy (`internal/handler/proxy.go`)

Go standart kütüphanesindeki `httputil.ReverseProxy` kullanılarak implement edilmiştir. Her downstream servis için ayrı bir proxy instance'ı oluşturulur.

```go
func NewProxy(target string) http.Handler {
    url, _ := url.Parse(target)
    proxy := httputil.NewSingleHostReverseProxy(url)
    proxy.Director = func(req *http.Request) {
        req.URL.Scheme = url.Scheme
        req.URL.Host   = url.Host
        req.Host       = url.Host
        req.Header.Set("ngrok-skip-browser-warning", "true")
    }
    return proxy
}
```

`Director` fonksiyonu her istekte çalışır ve hedef URL'yi set eder. `ngrok-skip-browser-warning` header'ı, ngrok üzerinden çalışan downstream servislerin browser uyarı sayfası döndürmesini engeller.

3.2 JWT Doğrulama (`internal/middleware/auth.go`)

Korumalı route group'larına uygulanan middleware. HMAC-SHA algoritmasıyla imzalanmış JWT token'larını doğrular.

```
token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (interface{}, error) {
    if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
        return nil, jwt.ErrSignatureInvalid
    }
    return []byte(secret), nil
})

// Doğrulama başarılıysa:
userIDFloat, _ := claims["user_id"].(float64)
userID := fmt.Sprintf("%d", int64(userIDFloat))
r.Header.Set("X-User-ID", userID)
```

Adım	Açıklama
Header kontrolü	`Authorization: Bearer <token>` formatı zorunlu
İmza doğrulama	Yalnızca HMAC yöntemi kabul edilir, diğerleri reddedilir
Claim çıkarma	Token içindeki `user_id` claim'i (float64) alınır, int64'e çevrilir
Header iletimi	`X-User-ID` olarak downstream servise iletilir
Context	`userID` değeri request context'ine de yazılır

Hatalı veya eksik token durumunda `401 Unauthorized` + `{"error":"unauthorized"}` döner.

3.3 Rate Limiter (`internal/middleware/ratelimit.go`)

Token bucket algoritması kullanılarak implement edilmiştir. Her IP için ayrı bir `rate.Limiter` instance'ı tutulur.

Mevcut konfigürasyon: `NewRateLimiter(10, 30)` — saniyede 10 istek, burst 30.

Özellik	Detay
Algoritma	Token bucket (`golang.org/x/time/rate`)
Kapsam	IP bazlı, in-memory
IP tespiti	Önce `X-Forwarded-For`, yoksa `RemoteAddr`
Bellek yönetimi	Background goroutine her dakika çalışır; 3 dakikadır görülmeyen IP'ler silinir
Limit aşımı	`429 Too Many Requests` + `{"error":"too many requests"}`

`cleanupLoop` goroutine'i `sync.Mutex` ile koruma altında çalışır, uzun süreli kullanımda bellek sızıntısını önler.

3.4 Circuit Breaker (`internal/middleware/circuitbreaker.go`)

Her downstream servis için bağımsız bir circuit breaker instance'ı vardır (`market-data` ve `exchange`).

Durum Makinesi

```
          5. hata
Closed ──────────────► Open
  ▲                      │
  │       30 saniye      │
  │    ◄─────────────────┘
  │                    Half-Open
  │      başarılı istek  │
  └──────────────────────┘
```

Durum	Açıklama
`Closed`	Normal çalışma — istekler geçer
`Open`	Servis çöktü — tüm istekler direkt 503 döner
`Half-Open`	Test modu — bir istek geçirilir; başarılıysa `Closed`'a döner

Hata tespiti: Downstream servisin `5xx` HTTP status kodu dönmesi hata sayılır. `4xx` hatalar (client hataları) sayılmaz.

Open durumunda response:

```
{
  "error": "service temporarily unavailable",
  "service": "market-data"
}
```

Tüm durum geçişleri `sync.Mutex` ile thread-safe'tir.

3.5 Prometheus Metrikleri (`internal/middleware/metrics.go`)

Her HTTP isteği için Prometheus sayaçları ve histogramları günceller.

Metrik	Tip	Açıklama
`http_requests_total`	Counter	method + path + status etiketleriyle toplam istek sayısı
`http_request_duration_seconds`	Histogram	İstek süresi dağılımı

Metrik	Tip	Açıklama
`rate_limit_hits_total`	Counter	Rate limiter tarafından reddedilen istek sayısı
`circuit_breaker_open`	Gauge	CB açıksa 1, kapalıysa 0 (servis bazlı)

`GET /metrics` endpoint'i Prometheus scraper'ına bu verileri sunar.

3.6 Request Logger (`internal/middleware/logging.go`)

Her isteği şu formatta loglar:

```
[2026-04-06 17:00:00] GET /api/v1/quotes/BTCUSDT | status=200 | latency=12ms |
ip=127.0.0.1:54321 | user=abc123
```

JWT doğrulandıktan sonra `X-User-ID` header'ı varsa kullanıcı ID'si, yoksa `-` yazar.

3.7 Health Aggregation (`internal/handler/health.go`) (`internal/handler/health.go`)

`GET /health` endpoint'i downstream servislerin `/health` endpoint'lerini kontrol eder ve birleşik durum döner.

Her servis için **3 saniyelik timeout** uygulanır. Downstream servisler ngrok üzerinden çalıştığından her isteğe `ngrok-skip-browser-warning: true` header'ı eklenir. Herhangi bir servis `down` ise:

- `status` alanı `"degraded"` olur
- HTTP status `503 Service Unavailable` döner

```
{
  "status": "ok",
  "services": {
    "market-data": { "status": "ok", "latency": "45ms" },
    "exchange":    { "status": "down", "error": "connection refused" }
  }
}
```

3.8 WebSocket Proxy (`internal/handler/websocket.go`)

`github.com/gorilla/websocket` ile implement edilmiştir. İki yönlü (bidirectional) tünel açar.

```
Client ─── ws/wss ──► api-gateway ─── ws/wss ──► Upstream
                ◄──────────────────────────────────────
```

Bağlantı sırası:

1. Önce upstream'e bağlanılır — başarısız olursa client'a HTTP 502 dönülür
2. Ardından client HTTP→WebSocket upgrade edilir
3. İki goroutine (her yön için) `errc` channel üzerinden senkronize çalışır — biri kapanınca diğeri de durur

Scheme dönüşümü: HTTP target URL'leri otomatik olarak WebSocket scheme'ine çevrilir:

- `http://` → `ws://`
- `https://` → `wss://`

Header iletimi: Yalnızca `Authorization` ve `X-` prefix'li header'lar upstream'e iletilir. JWT middleware tarafından set edilen `X-User-ID` bu sayede downstream'e ulaşır.

TLS: `InsecureSkipVerify: true` — ngrok sertifikalarını doğrulamadan geçer. Production'da kapatılabilir.

Tampon boyutu: Her yönde 32 KB (`gorilla/websocket` upgrader buffer'ı) — orderbook gibi büyük mesajlar için yeterli.

Handshake timeout: Upstream bağlantısı için 10 saniye.

> `/ws/positions/*` JWT korumalıdır — WebSocket handshake'ten önce token doğrulanır.

---

4. Route Tablosu

Gateway Sistem Endpoint'leri

Method	Path	Auth	Açıklama
GET	`/health`	—	Downstream servis durumu aggregation
GET	`/swagger/*`	—	Swagger UI
GET	`/swagger/doc.yaml`	—	OpenAPI YAML tanım dosyası
GET	`/metrics`	—	Prometheus metrik endpoint'i

market-data — Public (JWT Gerektirmez)

Method	Path	Circuit Breaker
ANY	`/api/v1/quotes/*`	market-data CB
ANY	`/api/v1/history/*`	market-data CB
ANY	`/api/v1/ohlcv/*`	market-data CB
ANY	`/api/v1/compare/*`	market-data CB
ANY	`/api/v1/funding/*`	market-data CB
ANY	`/api/v1/funding-rate/*`	market-data CB
ANY	`/api/v1/spread/*`	market-data CB
ANY	`/api/v1/efficiency/*`	market-data CB

Method	Path	Circuit Breaker
ANY	/api/v1/orderbook/*	market-data CB
ANY	/api/v1/liquidity/*	market-data CB
ANY	/api/v1/slippage/*	market-data CB
ANY	/api/v1/rsi/*	market-data CB
ANY	/api/v1/news	market-data CB
ANY	/api/v1/ico-calendar	market-data CB
ANY	/api/v1/etf-flows	market-data CB
ANY	/api/v1/whale-alerts	market-data CB
ANY	/api/v1/fees	market-data CB
ANY	/api/v1/all-in-cost/*	market-data CB
ANY	/api/v1/wallet/*	market-data CB
WS	/ws	—
WS	/ws/quotes/*	—
WS	/ws/orderbook	—

exchange-service — Public (JWT Gerektirmez)

Method	Path	Circuit Breaker
ANY	/api/v1/auth/*	exchange CB

exchange-service — Protected (JWT Zorunlu)

Method	Path	Circuit Breaker
ANY	/positions/*	exchange CB
ANY	/api/v1/portfolio/*	exchange CB
ANY	/api/v1/pnl/*	exchange CB
ANY	/api/v1/trades/*	exchange CB
ANY	/api/v1/orders	exchange CB
ANY	/api/v1/orders/*	exchange CB
ANY	/api/v1/apikeys	exchange CB
ANY	/api/v1/apikeys/*	exchange CB
ANY	/api/v1/alerts	exchange CB

Method	Path	Circuit Breaker
ANY	/api/v1/alerts/*	exchange CB
ANY	/api/v1/dca/*	exchange CB
ANY	/api/v1/risk/*	exchange CB
ANY	/api/v1/position/*	exchange CB
ANY	/api/v1/users/*	exchange CB
GET	/api/v1/performance	exchange CB
GET	/api/v1/staking	exchange CB
ANY	/api/v1/staking/*	exchange CB
GET	/api/v1/stacks	exchange CB
WS	/ws/positions/*	—
WS	/api/v1/ws	—
WS	/api/v1/ws/*	—

---

5. Konfigürasyon

Değişken	Varsayılan	Açıklama
PORT	9000	Gateway'in dinlediği port
JWT_SECRET	—	JWT doğrulama anahtarı (zorunlu)
MARKET_DATA_URL	ngrok URL	market-data servisinin adresi
EXCHANGE_URL	https://contextured-tora-nontribally.ngrok-free.dev	exchange-service'in adresi
RATE_LIMIT_RPS	10	Rate limiter — saniyedeki istek limiti (override)
RATE_LIMIT_BURST	30	Rate limiter — burst kapasitesi (override)

JWT_SECRET set edilmezse korumalı tüm endpoint'ler 401 döner.

---

6. Dosya Yapısı

```
api-gateway/
├── cmd/
│   └── main.go                    # Uygulama giriş noktası — router kurulumu,
middleware zinciri, route tanımları
```

```
├── config/
│   └── config.go                  # Environment variable okuma ve varsayılan
değerler
├── docs/
│   └── swagger.yaml               # OpenAPI 3.0 — tüm endpoint tanımları,
parametreler, auth şeması
├── internal/
│   ├── handler/
│   │   ├── health.go              # GET /health — downstream servis durumu
aggregation
│   │   ├── proxy.go               # HTTP reverse proxy (httputil.ReverseProxy
wrapper)
│   │   └── websocket.go           # WebSocket bidirectional tünel
│   └── middleware/
│       ├── auth.go                # JWT doğrulama — token parse, X-User-ID
iletimi
│       ├── circuitbreaker.go      # Circuit breaker state machine (Closed / Open
/ Half-Open)
│       ├── circuitbreaker_test.go # Circuit breaker unit testleri
│       ├── metrics.go             # Prometheus metrikleri — istek sayısı, süre,
rate limit, CB durumu
│       ├── logging.go             # RequestLogger — method, path, status,
latency, IP, user loglama
│       └── ratelimit.go           # IP bazlı token bucket rate limiter
├── Dockerfile                     # Multi-stage build — Go binary derleme +
minimal runtime image
├── docker-compose.yml             # Gateway servisini environment variable'larla
ayağa kaldırır
├── go.mod                         # Go modül tanımı ve bağımlılık listesi
└── go.sum                         # Bağımlılık checksum'ları
```

---

7. Test Sonuçları

Aşağıdaki testler ngrok üzerinden gerçek ortamda gerçekleştirilmiştir.

7.1 Health Aggregation

```
curl https://<gateway-ngrok>/health
# → {
#     "status": "ok",
#     "services": {
#       "market-data": { "status": "ok", "latency": "38ms" },
#       "exchange":    { "status": "ok", "latency": "52ms" }
#     }
#   }
```

✓ 3 servis de "ok" döndü — servisler arası bağlantı doğrulandı

7.2 JWT Koruması

```
# Token olmadan
curl https://<gateway-ngrok>/positions/1
# → {"error":"unauthorized"}  HTTP 401

# Geçerli token ile
curl -H "Authorization: Bearer <token>" https://<gateway-ngrok>/positions/1
# → pozisyon listesi  HTTP 200
```

✓ Korumasız istek reddedildi, geçerli token ile geçti

7.3 Rate Limiting

```
# 31. istekte (burst 30 aşıldığında)
# → {"error":"too many requests"}  HTTP 429
```

✓ Token bucket davranışı doğrulandı

7.4 Circuit Breaker

```
# exchange-service kapalıyken
curl -H "Authorization: Bearer <token>" https://<gateway-ngrok>/positions/1
# → {
#     "error": "service temporarily unavailable",
#     "service": "exchange"
#   }  HTTP 503
```

✓ 5 art arda hata sonrası devre açıldı, 30 saniye sonra otomatik kapandı

---

8. Swagger UI Kullanım Kılavuzu

Gateway çalışırken Terminale go run ./cmd/main.go `http://localhost:9000/swagger/index.html` adresini tarayıcıda aç.

8.1 Public Endpoint Test Etmek

1. Listeden bir endpoint'e tıkla (örn. `GET /api/v1/quotes/{symbol}`)
2. **Try it out** butonuna bas
3. Parametre kutularını doldur (örn. `symbol` = `BTCUSDT`)
4. **Execute** bas — gerçek API yanıtı altta görünür

8.2 JWT Gerektiren Endpoint Test Etmek

1. Önce token al:
    - `POST /api/v1/auth/login` → Try it out → email/password gir → Execute

    - Response'dan `token` değerini kopyala

2. Sağ üstteki **Authorize** butonuna tıkla
    - Value kutusuna token'ı yapıştır → **Authorize** → Close
3. Kilit ikonu olan endpoint'lere artık otomatik JWT ile istek atılır

8.3 API Tanım Dosyası

`docs/swagger.yaml` — tüm endpoint tanımları burada. Yeni endpoint eklendiğinde bu dosyaya da ilgili path eklenmeli.

---

9. Bilinen Kısıtlamalar ve Sonraki Adımlar

#	Kısıtlama	Durum	Planlanan Çözüm
1	WebSocket proxy'de circuit breaker yok	Açık	WS bağlantıları CB'den geçirilmeli
2	Health check'ler sıralı çalışıyor	Açık	`sync.WaitGroup` ile paralel hale getir
3	Rate limiter restart'ta sıfırlanır	Açık	Redis backed rate limiting
4	JWT token revocation yok	Açık	Token invalidation / blacklist mekanizması
5	Distributed tracing yok	Açık	Jaeger / OpenTelemetry entegrasyonu

---

Tresaurio Platform — api-gateway v0.7.0